TEAM T043 · PS-069 TECHNICAL GUIDE

Scholarship Eligibility & Disbursement Engine

Complete Step-by-Step Code Explanation, Execution Walkthrough, Business Rules & Command Reference

Language: Python 3.10+ (Standard Library only)

Project Directory: `C:\Users\PC\Documents\T043-PS-069`

Input Data: `T043-PS-069-sample-data` (500 apps, 6 schemes)

Verification: 11 / 11 automated tests passing

1. Project Overview & Folder Structure

This back-end engine automates how a state scholarship cell screens applications across multiple scholarship schemes, resolves overlapping eligibility, ranks applicants per district with reservation quotas, disburses awards in three instalments with runtime checks, handles dropouts and waitlist replacements, and processes annual academic renewals.

```
T043-PS-069/
├── main.py                    # Entry point (CLI flags & launches the engine)
├── engine/
│   ├── __init__.py            # Marks engine/ as a Python package
│   ├── models.py              # 9 simple dataclasses holding all records (168 lines)
│   ├── loader.py              # CSV & command-file readers + date/money helpers (197 lines)
│   ├── rules.py               # Pure functions for rules BR-01 to BR-12 (420 lines)
│   └── engine.py              # ScholarshipEngine class & CSV report writer (845 lines)
├── tests/
│   └── test_engine.py         # 11 unit + end-to-end tests (651 lines)
├── T043-PS-069-sample-data/   # Input CSVs + commands.txt
├── output/                    # 5 generated output CSVs
├── DESIGN.md                  # Registered BR-01..BR-12 table & viva notes
├── README.md                  # Quick-start instructions
└── .gitignore                 # Excludes __pycache__/ and temporary files
```

> **Why is the code split into 4 files inside `engine/` ?**
> Splitting data structures (`models.py`), file reading (`loader.py`), rule math (`rules.py`), and workflow state (`engine.py`) keeps each file focused on one job. Beginners can read or test any rule in `rules.py` without touching CSV parsing or ledger printing.

2. How the Project Runs (Step-by-Step Execution Flow)

When you run `python main.py` , the program follows a strict, deterministic pipeline from start to finish:

Step 1 — CLI Startup (`main.py`, lines 9–41)

`argparse` reads optional flags `--data-dir` (defaults to `T043-PS-069-sample-data`), `--output-dir` (defaults to `output`), and `--quiet`. It creates a `ScholarshipEngine` object and calls `run_commands_file()` followed by `write_outputs()`.

Step 2 — Loading Input Datasets (`ScholarshipEngine.__init__`, `engine.py` lines 49–86)

Before any command runs, the engine loads 7 CSV files into memory using `loader.py`: `schemes.csv` (6 schemes), `institutions.csv` (40 colleges), `fee_structure.csv` (73 course fees), `applications.csv` (500 applications), `bank_accounts.csv` (500 bank rows), `attendance_confirmations.csv` (1,000 rows for RUN-2 & RUN-3), and `renewals.csv` (63 rows).

Step 3 — Executing `commands.txt` Line by Line (`execute_command`, lines 100–129)

In `T043-PS-069-sample-data/commands.txt`, 28 commands run in strict chronological order:

1. `RUN_SCREENING|2026-08-20 10:00:00` — Deduplicates applicants (`BR-12`) and screens every application against all 6 schemes (3,000 matrix rows).
2. `RUN_SELECTION|2026-08-25 10:00:00` — Picks each applicant's single best scheme (`BR-04`), ranks within each district, applies 5% disability & 30% women quotas (`BR-05`), marks excess applicants `WAITLISTED`, and initializes the 50%/30%/20% instalment schedule.
3. `PRINT_SANCTION|ALL` — Prints the initial sanction summary and ranked list.
4. `DISBURSE|RUN-1|2026-09-15` — Releases 50% of means award + full Rs 5,000 book grant to `SELECTED` applicants whose bank validation passes on `2026-09-15`.
5. `DROPOUT` (lines 5–6) — Cancels future instalments for `APL-0492` (day 30 → `RECOVERABLE`) and `APL-0253` (day 31 → stays `PAID`).
6. `BANK_UPDATE` (line 7) — Updates `APL-0425`'s bank details to a `SELF` Aadhaar-seeded account before RUN-2.
7. `CANCEL_AWARD` (line 8) — Cancels `APL-0120` in Tumakuru on `2027-01-28`, promoting first waitlisted applicant `APL-0357` with 4 months pro-rated maintenance and full tuition.
8. `DISBURSE|RUN-2|2027-02-01` & `DISBURSE|RUN-3|2027-05-31` — Evaluates both bank ownership/age on run date AND attendance (>= 75%), carries forward 1 deferred instalment, or cancels on 2 consecutive deferrals.
9. `RUN_RENEWALS|2027-28|2027-07-20` — Evaluates all 63 renewal requests in `renewals.csv`.
10. `PRINT_UTILISATION` & 16 `PRINT_LEDGER` commands — Prints scheme utilisation totals and individual applicant ledgers.

Step 4 — Exporting the 5 Binding CSV Reports (`write_outputs`, `engine.py` lines 701–845)

Writes `eligibility_matrix.csv` (3,000 rows), `sanction.csv` (500 rows), `disbursement_ledger.csv`, `renewal_decisions.csv` (63 rows), and `utilisation.csv` (6 rows) sorted deterministically and formatted to 2 decimal places.

3. Detailed File-by-File Code Walkthrough

3.1 `main.py` (Lines 1–45)

The entry point uses Python's built-in `argparse` module so the engine can be run either directly with default folders or during evaluation with any hidden dataset folder.

```
engine = ScholarshipEngine(data_dir=args.data_dir, quiet=args.quiet)
engine.run_commands_file()
engine.write_outputs(args.output_dir)
```

3.2 `engine/models.py` (Lines 1–168)

Uses Python's `@dataclass` decorator to create clean, readable containers instead of messy raw dictionaries:

Dataclass	Lines	Purpose
`Scheme`	L13–L28	Stores scheme rules: type, income ceiling, merit cut-off, eligible courses, tuition cap, monthly maintenance, book grant, district seat cap, renewal min marks, and optional `budget_cap`.
`Institution`	L32–L38	Stores institution code, name, district, and `recognised` boolean.
`Applicant`	L42–L72	Stores demographic, academic, income, certificate date, and central scholarship fields. Includes `identity_key` property returning normalized `(applicant_name, date_of_birth, parent_name)` for duplicate detection.
`BankAccount`	L76–L84	Stores account holder name, `SELF` / `PARENT` relation, IFSC, last 4 digits, and `aadhaar_seeded` flag.
`RenewalRequest`	L88–L96	Stores renewal academic year, `promoted` flag, previous-year marks, certificate date, and submission date.
`EligibilityResult`	L100–L112	Stores one cell of the screening matrix: whether eligible, list of failed rule IDs, and computed award breakup.
`SanctionRecord`	L116–L143	Stores final sanction status (`SELECTED`, `WAITLISTED`, `INELIGIBLE`, `DUPLICATE`, `CANCELLED`), district rank, reserved bucket, plus runtime instalment dues (`run1_due`, `run2_due`, `run3_due`), `deferred_amount`, and `consecutive_deferrals`.
`LedgerEntry`	L147–L157	Stores each instalment attempt: run ID, date, %, amount due, gate result, amount paid, and status.
`RenewalDecision`	L161–L168	Stores renewal decision (`RENEWED`, `ON_PROBATION`, `TERMINATED`, `REJECTED_CERT`), reason code, and `maintenance_pct`.

3.3 `engine/loader.py` (Lines 1–197)

Responsible for reading all CSV files and safely converting text strings into typed Python objects:

- `parse_bool` (L17–L19), `parse_date` (L22–L24), `parse_timestamp` (L27–L29): Convert CSV strings to `bool`, `date`, and `datetime`.

- `add_months(start_date, months)` (L32–L39): Exact calendar month addition using `calendar.monthrange` so 12-month, 24-month, and 18-year (`18 * 12` months) checks handle leap years and month lengths accurately without assuming 365 days.
- `round_money(value)` (L42–L44): Rounds monetary values cleanly to 2 decimal places (`round(float(value) + 1e-9, 2)`) to avoid binary floating-point `.005` drift.
- `round_half_up(value)` (L47–L49): Converts fractional quotas like `0.5` seats up to `1` via `int(value + 0.5)` .
- `load_schemes` through `load_renewals` (L52–L184): Read each CSV using `csv.DictReader` .
- `load_commands` (L187–L197): Reads `commands.txt` , skips blank lines and `#` comments, and splits each command on `|` .

3.4 `engine/rules.py` (Lines 1–420) — Business Rules BR-01 to BR-12

Every business rule is isolated as a small, pure function in `rules.py`:

Rule	Function & Lines	Step-by-Step Logic
BR-12 Duplicates	`deduplicate_applications` (L31–L72)	Sorts all applications chronologically by `(application_ts, application_id)`. Uses `identity_key = (name, dob, parent_name)`. First application per person stays active and is screened across all 6 schemes; later applications (same scheme or extra schemes) are marked `DUPLICATE`.
BR-01 Income & Cert	`get_income_factor_pct` (L75–L81) `is_income_cert_valid_at_application` (L84–L90)	Income `<= 1,00,000` → `100%`; `1,00,001–2,50,000` → `75%`; `> 2,50,000` up to scheme ceiling → `50%`. Above ceiling fails with `BR-01_INELIGIBLE_INCOME`. Certificate must satisfy `application_ts.date() < add_months(income_cert_date, 12)`, else `BR-01_INVALID_CERT`.
BR-02 Merit Cut-off	`get_effective_merit_cutoff` (L93–L107)	If `merit_cutoff_pct > 0`, grants relaxation: `SC/ST = 5`, `OBC = 3`, `disability = 3`. Total relaxation is capped at `min(5.0, category_relax + disability_relax)` so SC+disability stays `5`. Also checks `minority == true` for `MINORITY` schemes.
BR-03 Course & Inst	`screen_application_against_scheme` (L110–L200)	Verifies `applicant.course in scheme.eligible_courses` and `institution.recognised == True`. Computes `tuition = min(annual_tuition, tuition_cap) * factor` and `maintenance = (monthly_rate * 10) * factor` only when `institution.district != home_district`.
BR-10 Book Grant	`screen_application_against_scheme` (L179–L180)	Adds Rs `5,000.00` only if `year_of_study == 1`. Not scaled by income slab, not pro-rated, and not included in renewals.
BR-04 Precedence	`select_best_scheme` (L203–L218)	Submits central scholarship offset (`max(0, tuition + maintenance + book_grant - central_tuition_component)`). Picks highest `award_total`; ties broken by `MERIT_CUM_MEANS (1) > MEANS (2) > MERIT (3) > MINORITY (4)`, then `scheme_code`.
BR-05 Ranking & Quotas	`sort_district_applicants` (L221–L231) `allocate_district_seats` (L234–L289)	Ranks within `(scheme, district)` by `(-previous_exam_pct, family_income, application_ts, application_id)`. When `seats_per_district` is set (e.g. 7 for SCH-01): fills `round_half_up(0.05 * S)` disability seats first, then `round_half_up(0.30 * S)` women seats, then remaining/unfilled seats as `OPEN`. Extra ranks become `WAITLISTED`.

Rule	Function & Lines	Step-by-Step Logic
BR-07 Bank Gate	`validate_bank_account` (L292–L310)	Requires `aadhaar_seeded == True`. If `holder_relation == "SELF"`, holder name must match `applicant_name`. If `"PARENT"`, applicant must be under 18 on `run_date` (`run_date < add_months(dob, 18 * 12)`) and holder name must match `parent_name`.
BR-08 Dropout Guard	`is_dropout_recoverable` (L339–L344)	Returns `True` when `0 <= (dropout_date - last_paid_date).days <= 30` (so day 30 is recoverable, day 31 is not).
BR-11 Waitlist Proration	`months_remaining_in_academic_year` (L313–L336)	Maps Aug–May into academic months 1–10. A cancellation on `2027-01-28` (day > 1 of month 6) starts replacement from month 7 (Feb), yielding `10 - 7 + 1 = 4` months left.
BR-09 Renewals	`evaluate_renewal` (L347–L420)	1) If `not promoted` → `TERMINATED` (`NOT_PROMOTED`). 2) If `submitted_date >= add_months(income_cert_date, 24)` → `REJECTED_CERT`. 3) Marks `>= 50%` (`>= 45%` for SC/ST) → `RENEWED` (`100%` maintenance, fail count reset to 0). 4) 1st marks failure → `ON_PROBATION` (`50%` maintenance); 2nd failure → `TERMINATED`.

3.5 `engine/engine.py` (Lines 1–845) — Stateful Command Processing

`ScholarshipEngine` maintains the live state across commands as dates advance over the academic year:

- **`run_screening` (L131–L155):** Runs deduplication, then calls `screen_application_against_scheme` for every `(applicant, scheme)` pair sorted by ID.
- **`run_selection` (L157–L239) & `_initialize_instalments` (L241–L248):** Marks duplicates/ineligibles, selects each eligible applicant's best scheme, runs district seat allocation, and splits each `SELECTED` award:
  - `run1_due = 50% of (award_total - book_grant) + book_grant`
  - `run2_due = 30% of (award_total - book_grant)`
  - `run3_due = remaining 20%` (computed by subtraction so the three instalments always sum to `award_total` to the exact paisa).
- **`disburse` (L280–L398):** Evaluates runtime gates at each run date:
  - If already cancelled (dropout or prior 2nd deferral), logs a `CANCELLED` ledger row with `0.00` paid.
  - Adds any `deferred_amount` from the previous failed run to the current scheduled instalment.
  - Checks attendance `>= 75.0` (for `RUN-2` & `RUN-3`) and `validate_bank_account` (for all runs).
  - On `PASS`: pays `total_due`, resets `consecutive_deferrals = 0` and `deferred_amount = 0`.
  - On failure: increments `consecutive_deferrals`. First failure → `DEFERRED`; second consecutive failure → permanently `CANCELLED`.
- **`handle_dropout` (L400–L425):** Marks record `CANCELLED`, clears deferred balance, and walks backwards through `self.ledger` to flip the most recent `PAID` entry to `RECOVERABLE` if within 30 days.
- **`handle_bank_update` (L427–L451):** Updates `account_holder_name`, `holder_relation`, and `aadhaar_seeded` in place so the next `DISBURSE` re-evaluates cleanly.
- **`cancel_award` (L453–L506) & `_pick_waitlist_replacement` (L508–L553):** Cancels the applicant. If cancelled before `RUN-2`, finds the next waitlisted applicant in that district (preferring the freed reserved group if under quota), scales maintenance by `months_left / 10.0`, keeps tuition 100%, sets book grant to `0.00`, and splits remaining award 60% at `RUN-2` and 40% at `RUN-3`.
- **Stretch Goals:** `_apply_optional_budget_caps` (L250–L278) moves lowest-ranked selections to waitlist if a scheme specifies `budget_cap`; `handle_dbt_return` (L555–L572) re-queues failed DBT credits back to `deferred_amount`.
- **`run_renewals` (L574–L592):** Evaluates `renewals.csv` against the applicant's original sanctioned scheme (or `scheme_applied` fallback) and tracks consecutive marks failures across renewal years.

4. Planted Edge Cases Verified in Sample Data & Tests

ID	Scenario	Example Record / Test	Engine Behaviour
EC-01	Income exactly 1,00,000 & 2,50,001	`APL-0374`, `APL-0464`	`1,00,000` gets 100% slab; `2,50,001` gets 50% slab and fails SCH-01 ceiling.
EC-02	SC + disability, 5 pts below cut-off	`APL-0433`	Relaxations do not stack beyond 5 (`60 - 5 = 55.00`), so `55.00%` is eligible.
EC-03	Identical total award tie-break	`APL-0107`	SCH-01 & SCH-02 tie; `MERIT_CUM_MEANS` beats `MEANS`.

ID	Scenario	Example Record / Test	Engine Behaviour
EC-04	30% of 7 seats non-integer	Belagavi (SCH-01)	`round_half_up(0.30*7) = 2` women, `round_half_up(0.05*7) = 0` disability.
EC-05	Attendance exactly 75.00%	`APL-0496`	`75.00 >= 75.0` passes RUN-2 gate.
EC-06	Dropout on day 30 vs day 31	`APL-0492` , `APL-0253`	Day 30 (`2026-10-15`) → `RECOVERABLE` ; day 31 (`2026-10-16`) → `PAID` .
EC-07	Renewal 50.00% & 24-month cert	`APL-0150`	`2025-07-15` cert submitted `2027-07-15` → `REJECTED_CERT` .
EC-08	Waitlist promotion, 4 months left	`APL-0120` → `APL-0357`	Cancelled `2027-01-28` → 4 months (Feb–May) maintenance (`4/10`) + full tuition.
Traps	Duplicates, bank fix, 2 deferrals	`APL-0061/0109/0326` , `APL-0425` , `APL-0378`	Duplicates marked `DUPLICATE` ; `BANK_UPDATE` resets deferral streak; 2 consecutive deferrals cancel award.

5. Output CSV Reference & Commands

Output File	Columns (in exact binding order)
`output/eligibility_matrix.csv`	`application_id, scheme_code, eligible, rule_ids_failed, award_total`
`output/sanction.csv`	`application_id, scheme_code, status, district, rank, reserved_category, tuition, maintenance, book_grant, central_reduction, income_factor_pct, award_total`
`output/disbursement_ledger.csv`	`application_id, run_id, run_date, instalment_pct, amount_due, gate_result, amount_paid, status`
`output/renewal_decisions.csv`	`application_id, academic_year, decision, reason_code, maintenance_pct`
`output/utilisation.csv`	`scheme_code, selected, waitlisted, sanctioned_amount, disbursed_amount, recoverable_amount`

Commands to Run the Engine & Tests

```
# 1. Navigate to project directory
cd C:\Users\PC\Documents\T043-PS-069

# 2. Run on the sample data pack (prints reports + writes output CSVs)
python main.py

# 3. Run on a custom input/output directory
python main.py --data-dir "path\to\data" --output-dir "path\to\output" --quiet

# 4. Run all 11 automated tests
python -m unittest discover -s tests -v
```